Lab 9: Facial Recognition

Final Project Codebase

Javier Guapilla Diaz & McKinley Nhi Seecof Quevedo

Imports

These are the imports used to run our code. Note that we had a GPU available to us, therefore we used CUDA.

In [2]:
```
import matplotlib.pyplot as plt
import torch
import numpy as np
import torch.nn as nn
import os
from PIL import Image # For easy manipulation of images and stuff
from PIL import ImageEnhance
import random
from sklearn.model_selection import train_test_split
import torch.nn.functional as F
import torch.optim as optim
from torch.utils.data import TensorDataset, DataLoader #added dataloader so that random data is batched

d = torch.device('cuda')
```

Prepare Data

Loading In Data

Due to the lack of uniformity in the data (emotion representation), we chose to join test data and train data, then shuffle and resplit evenly with proportional representation of data. This was done in an attempt to proportionally represent each emotion class in the data.

We then split the "big data" into test (20%) and train (80%), and took (10%) of train for a validation set. Giving use the following subsets: $\mathcal{D}_{train}, \mathcal{D}_{test}, \mathcal{D}_{validation}$.

We also chose to manipulate our training data by adding additional permutated data with slight rotation, brightness shifts, and reflections, effectively quadrupling the size of our original $\mathcal{D}_{train}$.

While it might not be the most efficient way to do this, we chose to augment data this way as it was the most straight forward.

In [ ]:
```
folders = ["train", "test"]
emotions = ["angry", "disgust", "fear", "happy", "neutral", "sad", "surprise"]
path = r"C:\Users\jguap\OneDrive\Desktop\spr26\final_data\data\\"
# path = r"C:\Users\AlpacaAdmin\Documents\UW\Spring26\final-proj\phys417final\FER-2013" MCKINLEY PATH
IMG_SIZE = (48, 48)

all_data = []
all_labels = []

# loading data (ALL OF IT)
for f in folders:
    for i in range(len(emotions)):
        e = emotions[i]
        curr_path = os.path.join(path, f, e)
        for filename in os.listdir(curr_path):
            curr_image_path = os.path.join(curr_path, filename)
            curr_image = Image.open(curr_image_path).convert("L")
            curr_image = curr_image.resize(IMG_SIZE)
            all_data.append(np.array(curr_image))
            all_labels.append(i)

all_data = np.array(all_data)
all_labels = np.array(all_labels)

# train/test split
X_train_raw, test_data, y_train_raw, test_labels = train_test_split(
    all_data, all_labels, test_size=0.2, stratify=all_labels, random_state=42
)

# train/val split
X_train_raw, X_val_raw, y_train_raw, validation_labels = train_test_split(
    X_train_raw, y_train_raw, test_size=0.1, stratify=y_train_raw, random_state=2
)

# step 3: adding to training (the transformations)
train_data = []
train_labels = []

for img, label in zip(X_train_raw, y_train_raw):
    # Loading in image
    curr_image = Image.fromarray(img)
    train_data.append(np.array(curr_image))
    train_labels.append(label)
    # adding permutations
    train_data.append(np.array(curr_image.transpose(Image.FLIP_LEFT_RIGHT)))
    train_labels.append(label)
    angle = random.uniform(-10, 10)
    train_data.append(np.array(curr_image.rotate(angle)))
    train_labels.append(label)
    factor = random.uniform(0.8, 1.2)
    train_data.append(np.array(ImageEnhance.Brightness(curr_image).enhance(factor)))
    train_labels.append(label)

train_data = np.array(train_data)
```

```
train_labels = np.array(train_labels)
validation_data = X_val_raw

train_total = len(train_data)
```

In [3]:
```
print(train_data.shape)
print(test_data.shape)
print(validation_data.shape)
```

```
(103352, 48, 48)
(7178, 48, 48)
(2871, 48, 48)
```

"Standardizing":

We then scaled our images, taking into account that they are already $[0, 255]$, performing a simple transformation taking our images from $[0, 255] \rightarrow [0, 1]$.

In [ ]:
```
# Reshaping
train_data = np.reshape(train_data, (train_total, (48*48)))
test_data = np.reshape(test_data, (len(test_data), (48*48)))
validation_data = np.reshape(validation_data, (len(validation_data), 48*48))

# scaling
train_data = train_data.astype(np.float32) / 255.0
validation_data = validation_data.astype(np.float32) / 255.0
test_data = test_data.astype(np.float32) / 255.0

# Reshape train/validation/test sets to conform to PyTorch's (N, Channels, Height, Width) standard for CNNs
train_features = train_data.reshape(-1, 1, 48, 48) # Going back to 48X48 images for CNN
validation_features = validation_data.reshape(-1, 1, 48, 48) # ^^
test_features = test_data.reshape(-1,1,48,48) # ^^
```

In [ ]:
```
# just to check shape
print(train_features.shape)
print(validation_features.shape)
print(test_features.shape)
```

```
(103352, 1, 48, 48)
(2871, 1, 48, 48)
(7178, 1, 48, 48)
```

Converting to Tensors

In order to work with our GPU, we transferred both $\mathcal{D}_{validation}$ and $\mathcal{D}_{test}$ to our GPU device. Note that we did NOT move our $\mathcal{D}_{train}$ as we use DataLoader which uses CPU, and instead is done later in training.

In [ ]:
```
# Make them into tensors
train_inputs = torch.from_numpy(train_features).float() # converting to floats
train_targets = torch.from_numpy(train_labels).long() # converting to 64-bit integer

validation_inputs = torch.from_numpy(validation_features).float().to(d) # converting to floats
validation_targets = torch.from_numpy(validation_labels).long().to(d) # converting to 64-bit integer

testing_inputs = torch.from_numpy(test_features).float().to(d) # converting to floats
testing_targets = torch.from_numpy(test_labels).long().to(d) # converting to 64-bit integer
```

Define Model

This is our model. In short, it is made up of 3 Convolutional Blocks followed by a 3 layer fully connected neural network that funnels down to our 7 emotions.

In [ ]:
```
class myCNNModel(torch.nn.Module):

    def __init__(self):

        super(myCNNModel, self).__init__()

        # Convolution followed by ReLU activation and a Maxpool
        self.cnn1 = nn.Conv2d(in_channels=1, out_channels=32,
                              kernel_size=3, stride=1, padding=1)
        self.relu1 = nn.ReLU()
        self.maxpool1 = nn.MaxPool2d(kernel_size=2)

        # Convolution followed by ReLU activation and a Maxpool
        self.cnn2 = nn.Conv2d(in_channels=32, out_channels=64,
                              kernel_size=3, stride=1, padding=1)
        self.relu2 = nn.ReLU()
        self.maxpool2 = nn.MaxPool2d(kernel_size=2)

        # Convolution followed by ReLU, maxpool and batch norm
        self.cnn3 = nn.Conv2d(in_channels=64, out_channels=128, kernel_size=3, stride=1, padding=1)
        self.relu3 = nn.ReLU()
        self.maxpool3 = nn.MaxPool2d(kernel_size=2)
        self.bn3 = nn.BatchNorm2d(128)

        # Squishing down to fully connected chunk.
        # self.fc1 = nn.Linear(128*6*6, 256)
        # self.fc2 = nn.Linear(256, 7)
        self.fc1 = nn.Linear(128*6*6, 256)
        self.fc3 = nn.Linear(256, 64)
        self.fc2 = nn.Linear(64, 7)

        # Batch normalizations used:
        self.bn = nn.BatchNorm1d(256)
        self.bn1 = nn.BatchNorm2d(32)
        self.bn2 = nn.BatchNorm2d(64)
        self.bn4 = nn.BatchNorm1d(64)

        # dropouts
        self.dropout = nn.Dropout(0.5)
        self.dropout_conv = nn.Dropout(0.25)

    def forward(self, x):
```

```
        # Convolution, batch normalization, ReLU, Maxpool
        out = self.cnn1(x)
        out = self.bn1(out)
        out = self.relu1(out)
        out = self.maxpool1(out)
        #out = self.dropout_conv(out) # *

        # Convolution, batch normalization, ReLU, Maxpool
        out = self.cnn2(out)
        out = self.bn2(out)
        out = self.relu2(out)
        out = self.maxpool2(out)
        out = self.dropout_conv(out) # *

        # ^ same as before
        out = self.cnn3(out)
        out = self.bn3(out)
        out = self.relu3(out)
        out = self.maxpool3(out)
        out = self.dropout_conv(out) # * Added dropouts bc of overfitting

        # NOTE: We go from 4068 -> 256 -> 64 -> 7
        # Flattening out our output
        out = out.view(out.size(0), -1)

        # Using a fully connected layer for the rest
        out = self.fc1(out)
        out = self.bn(out)
        out = F.relu(out)

        out = self.dropout(out)
        out = self.fc3(out)
        out = self.bn4(out)
        out = F.relu(out)

        out = self.dropout(out)
        out = self.fc2(out)

        return out
```

Define Hyperparameters

In [ ]:
```
# Fix the random seed so that model performance is reproducible
torch.manual_seed(55)

# Initializing model
model = myCNNModel()
model = model.to(d) # Bc I wanted to use my gpu, also "sent" model to GPU to work w/ tensors from before.

# Defining learning rate, epoch and batchsize for mini-batch gradient
learning_rate = 0.001
epochs = 25

## Mini-batch set up, splitting up the tensors from before
batchsize = 64
train_dataset = TensorDataset(train_inputs, train_targets)
val_dataset = TensorDataset(validation_inputs, validation_targets)

# NOTE: num_workers is for threads, and pin_memory is for locking memory
# kinda like in 483, since we want to read fast.
train_loader = DataLoader(train_dataset, batch_size=batchsize, shuffle=True, pin_memory=True, num_workers=2)
validation_loader = DataLoader(val_dataset,batch_size=batchsize, shuffle=False,  pin_memory=True, num_workers=2)

# Define loss function and optimizer
loss_func = nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(model.parameters(), lr=learning_rate)
scheduler = optim.lr_scheduler.ReduceLROnPlateau(optimizer, mode='min', factor=0.5, patience=5, min_lr=1e-6)

model # just to check the model I am using
```

Out[ ]:
```
myCNNModel(
  (cnn1): Conv2d(1, 32, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
  (relu1): ReLU()
  (maxpool1): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
  (cnn2): Conv2d(32, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
  (relu2): ReLU()
  (maxpool2): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
  (cnn3): Conv2d(64, 128, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
  (relu3): ReLU()
  (maxpool3): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
  (bn3): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
  (fc1): Linear(in_features=4608, out_features=256, bias=True)
  (fc3): Linear(in_features=256, out_features=64, bias=True)
  (fc2): Linear(in_features=64, out_features=7, bias=True)
  (bn): BatchNorm1d(256, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
  (bn1): BatchNorm2d(32, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
  (bn2): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
  (bn4): BatchNorm1d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
  (dropout): Dropout(p=0.5, inplace=False)
  (dropout_conv): Dropout(p=0.25, inplace=False)
)
```

Identify Tracked Values

In [10]:
```
# Placeholders for training loss and validation accuracy during training
train_loss_list = []
validation_loss_list = []
validation_accuracy_list = []
```

Train Model

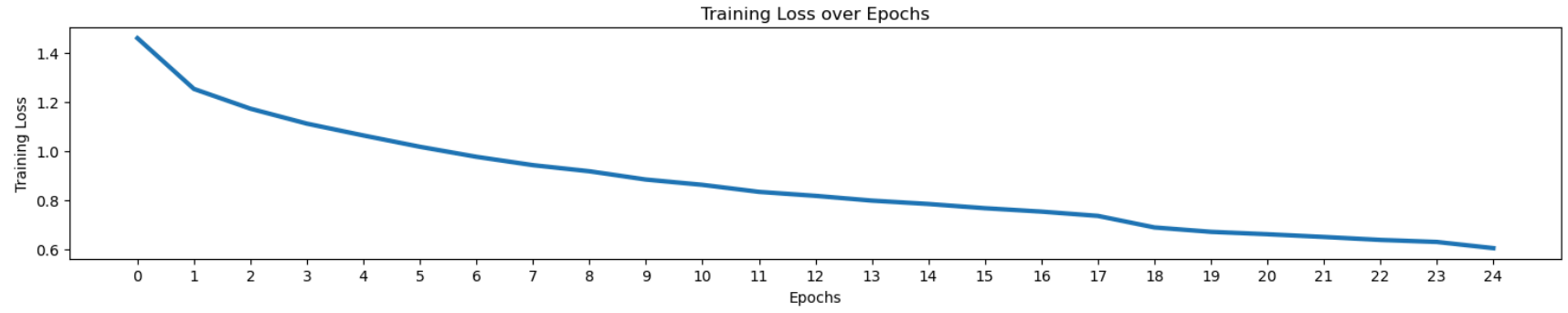
In [ ]:
```
import tqdm #For loading bar

# Actual training of the model:
for epoch in tqdm.trange(epochs):

    model.train() # I was told in a previous class doing this is good practice.
    e_loss = 0.0 # the average loss for this epoch bc I use mini-batch

    # Working w the batches
    # for k in range(batch_split_num):
    for features, targets in train_loader:
        # set up:
        # using batch k to train on
        features_k = features.to(d) # Moving to GPU now
        targets_k = targets.to(d) # ^

        # training
        optimizer.zero_grad()
        train_batch_outputs = model(features_k)
        loss = loss_func(train_batch_outputs, targets_k)
        e_loss += loss.item()

        # moving the optimizer along
        loss.backward()
        optimizer.step()

    epoch_avg_loss = e_loss / len(train_loader)
    train_loss_list.append(epoch_avg_loss)

    # Compute Validation Accuracy ----------------------------------------------------------------------
    model.eval() # Once again, I was told putting the model explicitly into eval mode was good practice
    with torch.no_grad(): # Telling PyTorch we aren't passing inputs to network for training purpose

        validation_outputs = model(validation_inputs) # Getting validation scores
        val_loss = loss_func(validation_outputs, validation_targets)
        validation_loss_list.append(val_loss.item())

        correct = (torch.argmax(validation_outputs, dim=1) ==
                   validation_targets).type(torch.FloatTensor) # converting validation scores to labels and
                                                               # counting the # of correctly labeled samples

        validation_accuracy_list.append(correct.mean()) # converting # of samples correct to percentage
        scheduler.step(val_loss.item())
```

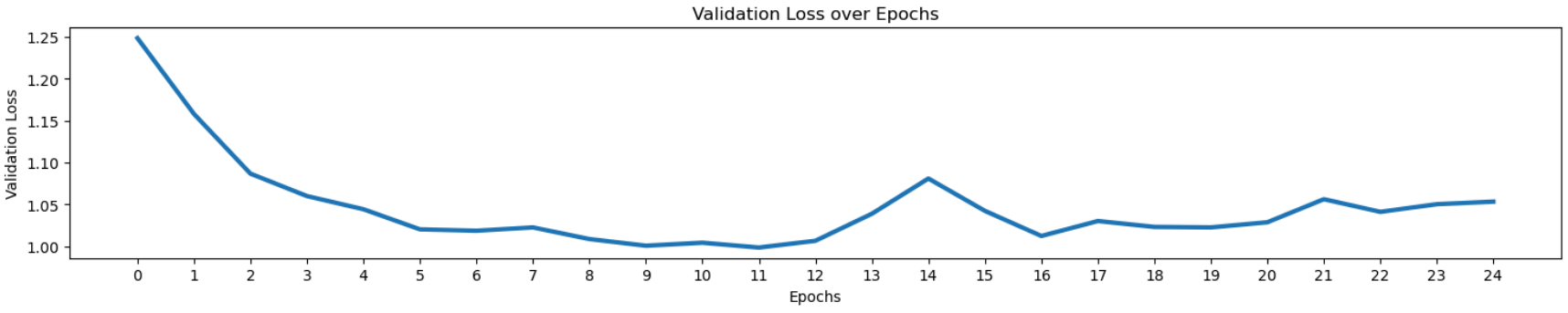
```
100%|██████████| 25/25 [09:14<00:00, 22.17s/it]
```

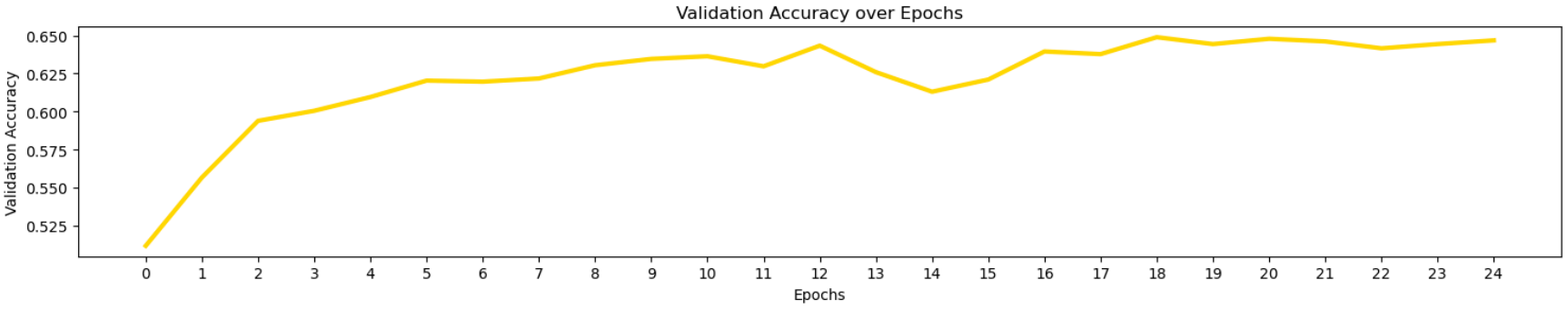
Visualize & Evaluate Model

Validation

In [12]:
```
# Visualizing training loss and validation set accuracy
plt.figure(figsize = (15, 9))
num_to = epochs
xt = np.arange(num_to)

# Plot for Training Loss
plt.subplot(3, 1, 1)
plt.plot(train_loss_list[:num_to], linewidth = 3)
plt.title("Training Loss over Epochs")
plt.ylabel("Training Loss")
plt.xlabel("Epochs")
plt.xticks(xt)

plt.subplot(3, 1, 2)
plt.plot(validation_loss_list[:num_to], linewidth = 3)
plt.title("Validation Loss over Epochs")
plt.ylabel("Validation Loss")
plt.xlabel("Epochs")
plt.xticks(xt)

# Accuracy for validation set
plt.subplot(3, 1, 3)
plt.plot(validation_accuracy_list[:num_to], linewidth = 3, color = 'gold')
plt.title("Validation Accuracy over Epochs")
plt.ylabel("Validation Accuracy")
plt.xlabel("Epochs")
plt.xticks(xt)
plt.tight_layout() #googled for overlapping labels
plt.show()
```







Test

Here, I wrote it so that we got both overall performance and by class.

In [ ]:
```python
# Compute the testing accuracy of all classes.
model.eval() # good practice again

print("--- Accuracy of Model ---\n")
with torch.no_grad():
    testing_inputs = testing_inputs.to(d)
    testing_targets = testing_targets.to(d)

    # testing model on test samples to get predicted values
    y_pred_test = model(testing_inputs)

    # Use the same technique as above to compute the testing classification accuracy
    predicted_targets = torch.argmax(y_pred_test, dim=1)
    correct = (predicted_targets == testing_targets).type(torch.FloatTensor)

    print("Testing Accuracy: " + str(correct.mean().numpy()*100) + '%\n') # printing the percentage

## NOTE: This is per Class VVVV
tt = testing_targets.cpu().numpy() #these are my true labels
pt = predicted_targets.cpu().numpy() # these are the predicted labels

# List of the different items we classified
c = ["angry", "disgust", "fear", "happy", "neutral", "sad", "surprise"]

print("--- Accuracy By Class ---\n")

# Looping over different classes/targets
for i in range(7):
    # Finding the accuracy:
    idx_k = np.where(tt==i) # finding true labels' indexs
    predicted_k = pt[idx_k] # using idx_k to find the predicted labels
    true_k = tt[idx_k] # getting the true labels
    percent_k = np.mean(predicted_k == true_k) # comparing true vs predicted

    # printing and formatting
    print("Accuracy of", c[i]+":", f"{percent_k*100:.2f}%")
    # ^ I used an f-string cuz it made formatting the percentage nicer
```

```
--- Accuracy of Model ---

Testing Accuracy: 65.00418%

--- Accuracy By Class ---

Accuracy of angry: 52.98%
Accuracy of disgust: 54.13%
Accuracy of fear: 41.21%
Accuracy of happy: 85.37%
Accuracy of neutral: 61.05%
Accuracy of sad: 58.80%
Accuracy of surprise: 81.62%
```

Printing Out Images (Actual VS Predicted)

In [16]:
```python
test_data_display = (test_features * 255.0).astype(np.uint8).reshape(-1, 48, 48)
fig, axes = plt.subplots(1, 7, figsize=(14, 3))

for i in range(7):
    idx = np.where(tt == i)[0][0]  # first instance of class i

    axes[i].imshow(test_data_display[idx], cmap="gray")
    axes[i].set_title(f"True: {c[tt[idx]]}\nPred: {c[pt[idx]]}", fontsize=8)
    axes[i].axis("off")
```

```
plt.tight_layout()
plt.show()
```

This saved the model that had the performance of just slightly $\geq 65\%$ testing accuracy.

In [ ]:
```
# Saving the model so it can be loaded for use later
#torch.save(model.state_dict(), "model3.pth")
```

LIVE COMPONENT

Here, we simply import the weights found for our best model (65%)

In [3]:
```
import cv2
import time
from ultralytics import YOLO
```

Below is the same model used above, we just renamed it and rewrote it to load in our saved model.

In [ ]:
```
# --- PyTorch Model Definition ---
class FacialExpressionCNN(torch.nn.Module):

    def __init__(self):

        super(FacialExpressionCNN, self).__init__()

        # Convolution followed by ReLU activation and a Maxpool
        self.cnn1 = nn.Conv2d(in_channels=1, out_channels=32,
                              kernel_size=3, stride=1, padding=1)
        self.relu1 = nn.ReLU()
        self.maxpool1 = nn.MaxPool2d(kernel_size=2)

        # Convolution followed by ReLU activation and a Maxpool
        self.cnn2 = nn.Conv2d(in_channels=32, out_channels=64,
                              kernel_size=3, stride=1, padding=1)
        self.relu2 = nn.ReLU()
        self.maxpool2 = nn.MaxPool2d(kernel_size=2)

        # added
        self.cnn3 = nn.Conv2d(in_channels=64, out_channels=128, kernel_size=3, stride=1, padding=1)
        self.relu3 = nn.ReLU()
        self.maxpool3 = nn.MaxPool2d(kernel_size=2)
        self.bn3 = nn.BatchNorm2d(128)

        # Squishing down to fully connected chunk.
        self.fc1 = nn.Linear(128*6*6, 256)
        self.fc3 = nn.Linear(256, 64)
        self.fc2 = nn.Linear(64, 7)

        # Batch normalizations used:
        self.bn = nn.BatchNorm1d(256)
        self.bn1 = nn.BatchNorm2d(32)
        self.bn2 = nn.BatchNorm2d(64)
        self.bn4 = nn.BatchNorm1d(64)

        # dropouts
        self.dropout = nn.Dropout(0.5)
        self.dropout_conv = nn.Dropout(0.25)

    def forward(self, x):

        # Convolution, batch normalization, ReLU, Maxpool
        out = self.cnn1(x)
        out = self.bn1(out)
        out = self.relu1(out)
        out = self.maxpool1(out)

        # Convolution, batch normalization, ReLU, Maxpool
        out = self.cnn2(out)
        out = self.bn2(out)
        out = self.relu2(out)
        out = self.maxpool2(out)
        out = self.dropout_conv(out)

        # ^ same as before
        out = self.cnn3(out)
        out = self.bn3(out)
        out = self.relu3(out)
        out = self.maxpool3(out)
        out = self.dropout_conv(out)

        # Flattening out our output
        out = out.view(out.size(0), -1)

        # Using a fully connected layer for the rest
        out = self.fc1(out)
        out = self.bn(out)
        out = F.relu(out)

        out = self.dropout(out)
        out = self.fc3(out)
        out = self.bn4(out)
        out = F.relu(out)
```

```
        out = self.dropout(out)
        out = self.fc2(out)

        return out

# --- Initialization ---
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

# This model helped is with "face searching" and allowed us to find faces in frame.
face_model = YOLO("yolov11n-face.pt")

# Loading in our model and setting it to evaluation mode.
emotion_model = FacialExpressionCNN().to(device)
emotion_model.load_state_dict(torch.load("model2.pth", map_location=device))
emotion_model.eval()

emotion_dict = {0: "Angry", 1: "Disgusted", 2: "Fearful", 3: "Happy", 4: "Neutral", 5: "Sad", 6: "Surprised"}

# THIS ANALYZES LIVE
def analyze_emotions():
    # Opens up the camera
    cap = cv2.VideoCapture(0)

    while cap.isOpened():
        ret, frame = cap.read()
        if not ret:
            break

        # detect faces
        results = face_model(frame)

        for r in results:
            for box in r.boxes:
                # get the boxes for each face
                x1, y1, x2, y2 = map(int, box.xyxy[0])
                cv2.rectangle(frame, (x1, y1), (x2, y2), (255, 0, 0), 2)

                # Pre-processing (OpenCV -> PyTorch Tensor)
                face = frame[y1:y2, x1:x2]
                if face.size == 0:
                    continue

                # Grayscale and resize
                face_gray = cv2.cvtColor(face, cv2.COLOR_BGR2GRAY)
                face_resized = cv2.resize(face_gray, (48, 48))

                # Convert to Tensor: (Batch, Channel, Height, Width)
                face_tensor = torch.from_numpy(face_resized).float().to(device)
                face_tensor = face_tensor.unsqueeze(0).unsqueeze(0) / 255.0

                # Prediction
                with torch.no_grad():
                    # model prediction
                    output = emotion_model(face_tensor)
                    prob = F.softmax(output, dim=1)
                    confidence, pred_idx = torch.max(prob, 1)

                    # label and prob
                    max_val = pred_idx.item()
                    emotion = emotion_dict[max_val]
                    conf_val = confidence.item()

                # putting label on persons face
                cv2.putText(frame, f"{emotion} ({conf_val:.2f})", (x1, y1 - 10),
                            cv2.FONT_HERSHEY_SIMPLEX, 0.8, (255, 255, 255), 2)

        # Shows the results in the current frame and option to quit.
        cv2.imshow("PyTorch Emotion Monitor", frame)
        if cv2.waitKey(1) & 0xFF == ord("q"):
            break

    cap.release()
    cv2.destroyAllWindows()

if __name__ == "__main__":
    analyze_emotions()
```

```
0: 480x640 (no detections), 154.2ms
Speed: 33.0ms preprocess, 154.2ms inference, 2.9ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 (no detections), 56.8ms
Speed: 3.5ms preprocess, 56.8ms inference, 2.8ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 (no detections), 56.7ms
Speed: 3.2ms preprocess, 56.7ms inference, 2.2ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 1 face, 62.3ms
Speed: 3.4ms preprocess, 62.3ms inference, 31.4ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 1 face, 49.5ms
Speed: 3.9ms preprocess, 49.5ms inference, 5.5ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 1 face, 49.0ms
Speed: 2.5ms preprocess, 49.0ms inference, 5.0ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 1 face, 61.1ms
Speed: 3.2ms preprocess, 61.1ms inference, 4.6ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 1 face, 51.4ms
Speed: 2.9ms preprocess, 51.4ms inference, 6.4ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 1 face, 48.4ms
Speed: 3.3ms preprocess, 48.4ms inference, 4.5ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 1 face, 47.3ms
Speed: 3.6ms preprocess, 47.3ms inference, 4.8ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 2 faces, 47.2ms
Speed: 4.0ms preprocess, 47.2ms inference, 23.0ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 2 faces, 51.9ms
Speed: 4.5ms preprocess, 51.9ms inference, 7.4ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 2 faces, 41.9ms
Speed: 4.0ms preprocess, 41.9ms inference, 4.3ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 2 faces, 55.4ms
Speed: 3.7ms preprocess, 55.4ms inference, 4.8ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 1 face, 56.6ms
Speed: 3.4ms preprocess, 56.6ms inference, 4.8ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 1 face, 45.7ms
Speed: 3.8ms preprocess, 45.7ms inference, 5.1ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 1 face, 59.7ms
Speed: 4.1ms preprocess, 59.7ms inference, 4.4ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 1 face, 42.4ms
Speed: 4.2ms preprocess, 42.4ms inference, 6.6ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 1 face, 52.0ms
Speed: 3.5ms preprocess, 52.0ms inference, 5.2ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 1 face, 40.5ms
Speed: 4.4ms preprocess, 40.5ms inference, 4.9ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 1 face, 58.4ms
Speed: 4.4ms preprocess, 58.4ms inference, 4.3ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 1 face, 43.5ms
Speed: 4.2ms preprocess, 43.5ms inference, 4.6ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 1 face, 38.1ms
Speed: 3.9ms preprocess, 38.1ms inference, 5.3ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 1 face, 49.1ms
Speed: 5.3ms preprocess, 49.1ms inference, 4.9ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 1 face, 63.8ms
Speed: 4.0ms preprocess, 63.8ms inference, 6.0ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 1 face, 52.8ms
Speed: 4.2ms preprocess, 52.8ms inference, 5.3ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 1 face, 40.2ms
Speed: 3.5ms preprocess, 40.2ms inference, 6.0ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 1 face, 37.7ms
Speed: 4.3ms preprocess, 37.7ms inference, 5.2ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 1 face, 39.1ms
Speed: 4.7ms preprocess, 39.1ms inference, 4.9ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 1 face, 47.3ms
Speed: 5.8ms preprocess, 47.3ms inference, 7.0ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 1 face, 51.5ms
Speed: 4.0ms preprocess, 51.5ms inference, 8.4ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 1 face, 58.9ms
Speed: 4.8ms preprocess, 58.9ms inference, 5.8ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 1 face, 62.6ms
Speed: 4.5ms preprocess, 62.6ms inference, 5.4ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 1 face, 44.6ms
Speed: 3.5ms preprocess, 44.6ms inference, 4.2ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 1 face, 47.5ms
Speed: 4.8ms preprocess, 47.5ms inference, 5.5ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 1 face, 48.6ms
```

```
Speed: 5.1ms preprocess, 48.6ms inference, 4.7ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 1 face, 37.8ms
Speed: 3.3ms preprocess, 37.8ms inference, 5.3ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 1 face, 66.9ms
Speed: 8.1ms preprocess, 66.9ms inference, 7.0ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 1 face, 45.6ms
Speed: 4.8ms preprocess, 45.6ms inference, 5.2ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 1 face, 62.0ms
Speed: 3.9ms preprocess, 62.0ms inference, 4.7ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 1 face, 47.2ms
Speed: 3.6ms preprocess, 47.2ms inference, 4.6ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 1 face, 52.6ms
Speed: 4.0ms preprocess, 52.6ms inference, 7.6ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 1 face, 41.2ms
Speed: 3.5ms preprocess, 41.2ms inference, 5.1ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 1 face, 34.2ms
Speed: 3.6ms preprocess, 34.2ms inference, 4.0ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 1 face, 38.5ms
Speed: 3.6ms preprocess, 38.5ms inference, 6.0ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 1 face, 41.4ms
Speed: 3.4ms preprocess, 41.4ms inference, 5.0ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 1 face, 40.9ms
Speed: 2.7ms preprocess, 40.9ms inference, 5.7ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 1 face, 57.8ms
Speed: 3.2ms preprocess, 57.8ms inference, 5.2ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 1 face, 60.8ms
Speed: 3.7ms preprocess, 60.8ms inference, 6.2ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 1 face, 47.8ms
Speed: 3.2ms preprocess, 47.8ms inference, 5.0ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 1 face, 47.0ms
Speed: 3.8ms preprocess, 47.0ms inference, 5.4ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 1 face, 56.0ms
Speed: 3.0ms preprocess, 56.0ms inference, 6.1ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 1 face, 53.1ms
Speed: 3.7ms preprocess, 53.1ms inference, 4.9ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 1 face, 42.0ms
Speed: 3.5ms preprocess, 42.0ms inference, 4.3ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 1 face, 45.7ms
Speed: 3.6ms preprocess, 45.7ms inference, 4.7ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 1 face, 43.6ms
Speed: 2.8ms preprocess, 43.6ms inference, 4.8ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 1 face, 52.3ms
Speed: 3.9ms preprocess, 52.3ms inference, 5.1ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 1 face, 51.5ms
Speed: 4.9ms preprocess, 51.5ms inference, 4.1ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 1 face, 52.1ms
Speed: 2.7ms preprocess, 52.1ms inference, 6.1ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 1 face, 43.7ms
Speed: 3.6ms preprocess, 43.7ms inference, 4.8ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 1 face, 62.9ms
Speed: 3.3ms preprocess, 62.9ms inference, 7.1ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 1 face, 80.6ms
Speed: 3.7ms preprocess, 80.6ms inference, 9.2ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 1 face, 76.5ms
Speed: 4.4ms preprocess, 76.5ms inference, 8.0ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 1 face, 37.6ms
Speed: 3.4ms preprocess, 37.6ms inference, 5.7ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 1 face, 36.1ms
Speed: 4.2ms preprocess, 36.1ms inference, 4.9ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 1 face, 34.0ms
Speed: 4.6ms preprocess, 34.0ms inference, 4.5ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 1 face, 36.2ms
Speed: 3.5ms preprocess, 36.2ms inference, 4.5ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 1 face, 34.3ms
Speed: 3.2ms preprocess, 34.3ms inference, 4.5ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 1 face, 56.0ms
Speed: 3.8ms preprocess, 56.0ms inference, 11.2ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 1 face, 55.7ms
Speed: 2.8ms preprocess, 55.7ms inference, 4.9ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 1 face, 40.7ms
Speed: 3.3ms preprocess, 40.7ms inference, 5.0ms postprocess per image at shape (1, 3, 480, 640)
```

```
0: 480x640 1 face, 44.1ms
Speed: 3.5ms preprocess, 44.1ms inference, 4.7ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 1 face, 47.5ms
Speed: 3.3ms preprocess, 47.5ms inference, 6.9ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 1 face, 66.7ms
Speed: 4.2ms preprocess, 66.7ms inference, 5.9ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 1 face, 38.9ms
Speed: 4.7ms preprocess, 38.9ms inference, 4.6ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 1 face, 39.8ms
Speed: 2.8ms preprocess, 39.8ms inference, 5.1ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 1 face, 39.9ms
Speed: 2.5ms preprocess, 39.9ms inference, 4.8ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 1 face, 55.4ms
Speed: 3.1ms preprocess, 55.4ms inference, 4.5ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 1 face, 37.5ms
Speed: 3.6ms preprocess, 37.5ms inference, 4.8ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 1 face, 61.7ms
Speed: 3.0ms preprocess, 61.7ms inference, 5.9ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 1 face, 41.8ms
Speed: 3.8ms preprocess, 41.8ms inference, 4.7ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 1 face, 42.7ms
Speed: 3.4ms preprocess, 42.7ms inference, 4.6ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 1 face, 69.2ms
Speed: 3.4ms preprocess, 69.2ms inference, 11.2ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 1 face, 45.8ms
Speed: 3.8ms preprocess, 45.8ms inference, 5.3ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 1 face, 45.7ms
Speed: 3.4ms preprocess, 45.7ms inference, 8.2ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 1 face, 42.5ms
Speed: 3.7ms preprocess, 42.5ms inference, 4.7ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 1 face, 44.9ms
Speed: 3.0ms preprocess, 44.9ms inference, 4.6ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 1 face, 40.5ms
Speed: 4.4ms preprocess, 40.5ms inference, 4.3ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 1 face, 60.0ms
Speed: 5.4ms preprocess, 60.0ms inference, 7.7ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 1 face, 50.7ms
Speed: 3.8ms preprocess, 50.7ms inference, 4.3ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 1 face, 47.1ms
Speed: 4.3ms preprocess, 47.1ms inference, 7.5ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 1 face, 40.4ms
Speed: 3.8ms preprocess, 40.4ms inference, 4.7ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 1 face, 38.7ms
Speed: 3.6ms preprocess, 38.7ms inference, 5.0ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 1 face, 44.7ms
Speed: 4.6ms preprocess, 44.7ms inference, 4.8ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 1 face, 40.4ms
Speed: 3.4ms preprocess, 40.4ms inference, 4.8ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 1 face, 47.0ms
Speed: 3.0ms preprocess, 47.0ms inference, 4.9ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 1 face, 43.6ms
Speed: 4.5ms preprocess, 43.6ms inference, 7.5ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 1 face, 42.3ms
Speed: 2.8ms preprocess, 42.3ms inference, 5.7ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 1 face, 72.9ms
Speed: 3.4ms preprocess, 72.9ms inference, 8.7ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 1 face, 79.9ms
Speed: 3.8ms preprocess, 79.9ms inference, 7.8ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 1 face, 64.2ms
Speed: 5.9ms preprocess, 64.2ms inference, 5.9ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 1 face, 70.8ms
Speed: 5.5ms preprocess, 70.8ms inference, 6.6ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 1 face, 78.3ms
Speed: 5.4ms preprocess, 78.3ms inference, 5.7ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 1 face, 69.8ms
Speed: 3.4ms preprocess, 69.8ms inference, 8.7ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 1 face, 51.3ms
Speed: 3.9ms preprocess, 51.3ms inference, 4.4ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 1 face, 38.8ms
Speed: 3.4ms preprocess, 38.8ms inference, 4.5ms postprocess per image at shape (1, 3, 480, 640)
```

```
0: 480x640 1 face, 38.0ms
Speed: 3.8ms preprocess, 38.0ms inference, 4.3ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 1 face, 52.0ms
Speed: 3.6ms preprocess, 52.0ms inference, 5.2ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 1 face, 69.2ms
Speed: 3.7ms preprocess, 69.2ms inference, 6.3ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 1 face, 55.7ms
Speed: 4.0ms preprocess, 55.7ms inference, 7.4ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 1 face, 42.6ms
Speed: 3.4ms preprocess, 42.6ms inference, 4.6ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 1 face, 43.1ms
Speed: 3.1ms preprocess, 43.1ms inference, 6.6ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 1 face, 56.8ms
Speed: 3.3ms preprocess, 56.8ms inference, 4.7ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 1 face, 56.3ms
Speed: 3.5ms preprocess, 56.3ms inference, 7.6ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 1 face, 62.0ms
Speed: 3.1ms preprocess, 62.0ms inference, 5.1ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 1 face, 44.5ms
Speed: 3.4ms preprocess, 44.5ms inference, 4.4ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 1 face, 39.6ms
Speed: 2.6ms preprocess, 39.6ms inference, 5.3ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 1 face, 41.9ms
Speed: 4.1ms preprocess, 41.9ms inference, 6.0ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 1 face, 38.3ms
Speed: 3.8ms preprocess, 38.3ms inference, 4.8ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 1 face, 46.2ms
Speed: 2.4ms preprocess, 46.2ms inference, 7.4ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 1 face, 53.9ms
Speed: 3.3ms preprocess, 53.9ms inference, 5.8ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 1 face, 55.2ms
Speed: 3.1ms preprocess, 55.2ms inference, 6.0ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 1 face, 65.4ms
Speed: 3.8ms preprocess, 65.4ms inference, 6.1ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 1 face, 48.2ms
Speed: 3.3ms preprocess, 48.2ms inference, 4.9ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 1 face, 40.7ms
Speed: 3.4ms preprocess, 40.7ms inference, 4.6ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 1 face, 41.3ms
Speed: 3.9ms preprocess, 41.3ms inference, 5.0ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 1 face, 78.6ms
Speed: 5.6ms preprocess, 78.6ms inference, 8.5ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 1 face, 50.5ms
Speed: 4.5ms preprocess, 50.5ms inference, 6.2ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 1 face, 61.3ms
Speed: 3.8ms preprocess, 61.3ms inference, 7.7ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 1 face, 61.4ms
Speed: 4.0ms preprocess, 61.4ms inference, 6.9ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 1 face, 51.2ms
Speed: 4.3ms preprocess, 51.2ms inference, 6.2ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 1 face, 54.7ms
Speed: 3.6ms preprocess, 54.7ms inference, 5.1ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 1 face, 50.0ms
Speed: 4.4ms preprocess, 50.0ms inference, 4.7ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 1 face, 57.4ms
Speed: 2.7ms preprocess, 57.4ms inference, 4.8ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 1 face, 56.1ms
Speed: 4.2ms preprocess, 56.1ms inference, 5.2ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 1 face, 66.3ms
Speed: 3.3ms preprocess, 66.3ms inference, 10.5ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 1 face, 60.9ms
Speed: 4.0ms preprocess, 60.9ms inference, 6.2ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 1 face, 55.1ms
Speed: 4.4ms preprocess, 55.1ms inference, 5.8ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 1 face, 53.6ms
Speed: 4.8ms preprocess, 53.6ms inference, 6.7ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 1 face, 63.9ms
Speed: 3.9ms preprocess, 63.9ms inference, 5.2ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 1 face, 43.8ms
Speed: 3.2ms preprocess, 43.8ms inference, 7.7ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 1 face, 42.9ms
```

```
Speed: 2.8ms preprocess, 42.9ms inference, 5.2ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 1 face, 47.2ms
Speed: 2.7ms preprocess, 47.2ms inference, 5.9ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 1 face, 69.2ms
Speed: 3.1ms preprocess, 69.2ms inference, 4.4ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 1 face, 49.7ms
Speed: 2.7ms preprocess, 49.7ms inference, 4.9ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 1 face, 48.3ms
Speed: 2.9ms preprocess, 48.3ms inference, 6.6ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 1 face, 63.9ms
Speed: 3.8ms preprocess, 63.9ms inference, 6.0ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 1 face, 50.8ms
Speed: 3.6ms preprocess, 50.8ms inference, 6.1ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 1 face, 46.9ms
Speed: 2.5ms preprocess, 46.9ms inference, 5.3ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 1 face, 46.7ms
Speed: 3.2ms preprocess, 46.7ms inference, 5.0ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 1 face, 71.7ms
Speed: 4.4ms preprocess, 71.7ms inference, 5.5ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 1 face, 65.2ms
Speed: 3.4ms preprocess, 65.2ms inference, 7.5ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 1 face, 55.4ms
Speed: 3.1ms preprocess, 55.4ms inference, 9.4ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 1 face, 49.0ms
Speed: 2.9ms preprocess, 49.0ms inference, 4.8ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 1 face, 44.5ms
Speed: 3.4ms preprocess, 44.5ms inference, 5.7ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 1 face, 44.8ms
Speed: 3.0ms preprocess, 44.8ms inference, 4.8ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 1 face, 64.4ms
Speed: 4.1ms preprocess, 64.4ms inference, 6.6ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 1 face, 44.3ms
Speed: 3.9ms preprocess, 44.3ms inference, 4.3ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 1 face, 43.5ms
Speed: 3.5ms preprocess, 43.5ms inference, 4.8ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 1 face, 50.8ms
Speed: 3.4ms preprocess, 50.8ms inference, 5.5ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 1 face, 56.2ms
Speed: 3.4ms preprocess, 56.2ms inference, 5.1ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 2 faces, 50.2ms
Speed: 4.2ms preprocess, 50.2ms inference, 5.3ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 2 faces, 43.7ms
Speed: 3.4ms preprocess, 43.7ms inference, 4.9ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 2 faces, 50.3ms
Speed: 3.5ms preprocess, 50.3ms inference, 4.9ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 2 faces, 56.6ms
Speed: 3.6ms preprocess, 56.6ms inference, 5.9ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 2 faces, 52.7ms
Speed: 3.3ms preprocess, 52.7ms inference, 8.9ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 2 faces, 59.3ms
Speed: 4.7ms preprocess, 59.3ms inference, 9.0ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 2 faces, 72.8ms
Speed: 3.4ms preprocess, 72.8ms inference, 8.0ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 2 faces, 61.8ms
Speed: 4.9ms preprocess, 61.8ms inference, 6.3ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 2 faces, 43.8ms
Speed: 2.9ms preprocess, 43.8ms inference, 11.4ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 2 faces, 82.9ms
Speed: 3.4ms preprocess, 82.9ms inference, 10.8ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 2 faces, 36.2ms
Speed: 2.6ms preprocess, 36.2ms inference, 4.2ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 2 faces, 75.0ms
Speed: 3.6ms preprocess, 75.0ms inference, 11.0ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 2 faces, 52.5ms
Speed: 3.4ms preprocess, 52.5ms inference, 5.3ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 2 faces, 50.1ms
Speed: 5.0ms preprocess, 50.1ms inference, 6.2ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 2 faces, 62.4ms
Speed: 3.9ms preprocess, 62.4ms inference, 8.7ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 2 faces, 52.8ms
Speed: 5.7ms preprocess, 52.8ms inference, 4.9ms postprocess per image at shape (1, 3, 480, 640)
```

```
0: 480x640 2 faces, 48.1ms
Speed: 3.8ms preprocess, 48.1ms inference, 6.2ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 2 faces, 46.7ms
Speed: 3.4ms preprocess, 46.7ms inference, 7.0ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 2 faces, 35.4ms
Speed: 3.7ms preprocess, 35.4ms inference, 5.2ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 2 faces, 61.2ms
Speed: 3.1ms preprocess, 61.2ms inference, 7.2ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 2 faces, 40.8ms
Speed: 2.6ms preprocess, 40.8ms inference, 4.9ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 2 faces, 47.3ms
Speed: 4.0ms preprocess, 47.3ms inference, 5.0ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 2 faces, 45.4ms
Speed: 4.2ms preprocess, 45.4ms inference, 4.2ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 2 faces, 40.8ms
Speed: 4.3ms preprocess, 40.8ms inference, 4.4ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 2 faces, 36.3ms
Speed: 2.5ms preprocess, 36.3ms inference, 5.0ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 2 faces, 52.3ms
Speed: 3.0ms preprocess, 52.3ms inference, 5.7ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 2 faces, 44.8ms
Speed: 3.5ms preprocess, 44.8ms inference, 4.5ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 2 faces, 59.3ms
Speed: 3.6ms preprocess, 59.3ms inference, 5.0ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 2 faces, 39.7ms
Speed: 2.4ms preprocess, 39.7ms inference, 4.4ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 2 faces, 46.2ms
Speed: 2.8ms preprocess, 46.2ms inference, 5.3ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 2 faces, 37.2ms
Speed: 3.2ms preprocess, 37.2ms inference, 4.4ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 2 faces, 41.7ms
Speed: 4.0ms preprocess, 41.7ms inference, 4.5ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 2 faces, 50.7ms
Speed: 4.3ms preprocess, 50.7ms inference, 6.3ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 2 faces, 47.3ms
Speed: 3.0ms preprocess, 47.3ms inference, 4.8ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 2 faces, 49.5ms
Speed: 3.1ms preprocess, 49.5ms inference, 7.4ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 2 faces, 52.4ms
Speed: 4.1ms preprocess, 52.4ms inference, 5.7ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 2 faces, 49.1ms
Speed: 2.7ms preprocess, 49.1ms inference, 4.4ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 2 faces, 53.2ms
Speed: 3.4ms preprocess, 53.2ms inference, 5.5ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 2 faces, 47.6ms
Speed: 4.4ms preprocess, 47.6ms inference, 5.1ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 2 faces, 43.6ms
Speed: 3.6ms preprocess, 43.6ms inference, 7.1ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 2 faces, 37.1ms
Speed: 4.6ms preprocess, 37.1ms inference, 4.3ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 2 faces, 46.3ms
Speed: 4.4ms preprocess, 46.3ms inference, 6.4ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 2 faces, 40.0ms
Speed: 4.8ms preprocess, 40.0ms inference, 4.2ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 2 faces, 63.2ms
Speed: 3.2ms preprocess, 63.2ms inference, 7.7ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 2 faces, 56.0ms
Speed: 2.9ms preprocess, 56.0ms inference, 5.1ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 2 faces, 62.3ms
Speed: 3.5ms preprocess, 62.3ms inference, 5.3ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 2 faces, 53.2ms
Speed: 4.0ms preprocess, 53.2ms inference, 4.8ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 2 faces, 39.0ms
Speed: 3.1ms preprocess, 39.0ms inference, 6.3ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 2 faces, 45.6ms
Speed: 2.6ms preprocess, 45.6ms inference, 4.4ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 2 faces, 41.7ms
Speed: 3.7ms preprocess, 41.7ms inference, 4.6ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 2 faces, 41.5ms
Speed: 2.7ms preprocess, 41.5ms inference, 4.7ms postprocess per image at shape (1, 3, 480, 640)
```

```
0: 480x640 2 faces, 42.5ms
Speed: 2.6ms preprocess, 42.5ms inference, 7.9ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 2 faces, 36.9ms
Speed: 3.4ms preprocess, 36.9ms inference, 4.5ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 2 faces, 51.3ms
Speed: 2.7ms preprocess, 51.3ms inference, 6.4ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 2 faces, 38.9ms
Speed: 3.4ms preprocess, 38.9ms inference, 4.7ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 2 faces, 43.2ms
Speed: 3.2ms preprocess, 43.2ms inference, 4.5ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 2 faces, 40.9ms
Speed: 2.6ms preprocess, 40.9ms inference, 4.6ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 2 faces, 40.3ms
Speed: 2.6ms preprocess, 40.3ms inference, 5.5ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 2 faces, 47.9ms
Speed: 3.4ms preprocess, 47.9ms inference, 5.8ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 2 faces, 45.9ms
Speed: 3.0ms preprocess, 45.9ms inference, 7.1ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 1 face, 53.2ms
Speed: 3.7ms preprocess, 53.2ms inference, 5.4ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 1 face, 52.9ms
Speed: 2.9ms preprocess, 52.9ms inference, 4.3ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 1 face, 42.9ms
Speed: 6.0ms preprocess, 42.9ms inference, 5.9ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 1 face, 58.3ms
Speed: 4.4ms preprocess, 58.3ms inference, 4.7ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 1 face, 47.6ms
Speed: 4.4ms preprocess, 47.6ms inference, 4.8ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 1 face, 44.3ms
Speed: 3.7ms preprocess, 44.3ms inference, 4.4ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 1 face, 43.4ms
Speed: 4.9ms preprocess, 43.4ms inference, 7.6ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 1 face, 48.7ms
Speed: 3.5ms preprocess, 48.7ms inference, 5.3ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 1 face, 51.6ms
Speed: 4.6ms preprocess, 51.6ms inference, 7.5ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 1 face, 48.4ms
Speed: 3.6ms preprocess, 48.4ms inference, 6.3ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 1 face, 42.4ms
Speed: 4.5ms preprocess, 42.4ms inference, 5.6ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 1 face, 50.8ms
Speed: 5.3ms preprocess, 50.8ms inference, 5.2ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 1 face, 51.2ms
Speed: 3.9ms preprocess, 51.2ms inference, 7.2ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 1 face, 43.4ms
Speed: 4.9ms preprocess, 43.4ms inference, 5.5ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 1 face, 40.9ms
Speed: 2.9ms preprocess, 40.9ms inference, 5.2ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 1 face, 46.7ms
Speed: 3.4ms preprocess, 46.7ms inference, 6.9ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 1 face, 47.0ms
Speed: 4.0ms preprocess, 47.0ms inference, 4.4ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 1 face, 54.7ms
Speed: 4.8ms preprocess, 54.7ms inference, 5.1ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 1 face, 48.2ms
Speed: 3.5ms preprocess, 48.2ms inference, 4.3ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 1 face, 38.5ms
Speed: 2.9ms preprocess, 38.5ms inference, 5.7ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 1 face, 48.0ms
Speed: 3.5ms preprocess, 48.0ms inference, 4.9ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 1 face, 55.6ms
Speed: 3.0ms preprocess, 55.6ms inference, 6.8ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 1 face, 53.0ms
Speed: 3.2ms preprocess, 53.0ms inference, 6.6ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 1 face, 60.9ms
Speed: 4.3ms preprocess, 60.9ms inference, 4.1ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 1 face, 40.4ms
Speed: 4.1ms preprocess, 40.4ms inference, 5.5ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 1 face, 65.7ms
Speed: 3.5ms preprocess, 65.7ms inference, 5.2ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 1 face, 39.4ms
```

```
Speed: 3.6ms preprocess, 39.4ms inference, 5.0ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 1 face, 56.5ms
Speed: 3.6ms preprocess, 56.5ms inference, 5.2ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 1 face, 42.6ms
Speed: 3.2ms preprocess, 42.6ms inference, 6.9ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 1 face, 42.8ms
Speed: 2.9ms preprocess, 42.8ms inference, 5.1ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 1 face, 44.4ms
Speed: 3.3ms preprocess, 44.4ms inference, 5.1ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 1 face, 51.7ms
Speed: 3.3ms preprocess, 51.7ms inference, 7.2ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 1 face, 47.8ms
Speed: 3.9ms preprocess, 47.8ms inference, 4.8ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 1 face, 37.5ms
Speed: 2.9ms preprocess, 37.5ms inference, 4.5ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 1 face, 44.0ms
Speed: 3.6ms preprocess, 44.0ms inference, 6.9ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 1 face, 50.3ms
Speed: 3.8ms preprocess, 50.3ms inference, 6.8ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 1 face, 45.3ms
Speed: 4.2ms preprocess, 45.3ms inference, 5.4ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 1 face, 51.5ms
Speed: 4.1ms preprocess, 51.5ms inference, 5.9ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 1 face, 40.8ms
Speed: 4.6ms preprocess, 40.8ms inference, 4.6ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 1 face, 39.8ms
Speed: 3.7ms preprocess, 39.8ms inference, 5.4ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 1 face, 44.6ms
Speed: 5.3ms preprocess, 44.6ms inference, 7.0ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 1 face, 57.7ms
Speed: 3.2ms preprocess, 57.7ms inference, 6.5ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 1 face, 41.9ms
Speed: 4.3ms preprocess, 41.9ms inference, 4.5ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 1 face, 53.9ms
Speed: 2.8ms preprocess, 53.9ms inference, 5.5ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 1 face, 56.7ms
Speed: 3.6ms preprocess, 56.7ms inference, 4.3ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 1 face, 50.4ms
Speed: 3.0ms preprocess, 50.4ms inference, 4.8ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 1 face, 53.6ms
Speed: 9.6ms preprocess, 53.6ms inference, 9.4ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 1 face, 44.0ms
Speed: 3.0ms preprocess, 44.0ms inference, 5.0ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 1 face, 53.4ms
Speed: 3.7ms preprocess, 53.4ms inference, 6.3ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 1 face, 58.0ms
Speed: 3.4ms preprocess, 58.0ms inference, 6.2ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 1 face, 49.2ms
Speed: 4.9ms preprocess, 49.2ms inference, 4.6ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 1 face, 56.3ms
Speed: 3.5ms preprocess, 56.3ms inference, 6.8ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 1 face, 77.5ms
Speed: 4.2ms preprocess, 77.5ms inference, 7.7ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 1 face, 56.7ms
Speed: 3.5ms preprocess, 56.7ms inference, 5.1ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 1 face, 77.6ms
Speed: 4.2ms preprocess, 77.6ms inference, 10.4ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 1 face, 35.5ms
Speed: 3.5ms preprocess, 35.5ms inference, 4.7ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 1 face, 47.1ms
Speed: 3.3ms preprocess, 47.1ms inference, 4.9ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 1 face, 73.7ms
Speed: 3.4ms preprocess, 73.7ms inference, 10.1ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 1 face, 74.7ms
Speed: 3.3ms preprocess, 74.7ms inference, 9.7ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 1 face, 41.8ms
Speed: 3.8ms preprocess, 41.8ms inference, 4.5ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 1 face, 45.8ms
Speed: 3.8ms preprocess, 45.8ms inference, 4.6ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 1 face, 41.2ms
Speed: 3.8ms preprocess, 41.2ms inference, 5.0ms postprocess per image at shape (1, 3, 480, 640)
```